

2026 €€€ 非专业级别软件能力认证第一轮
(C5P-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 26 日 14:30~16:30

考生注意事项：

- 试题纸共有 14 页，答题纸共有 1 页，满分 100 分。本卷为自制模拟卷，不代表任何官方机构。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在 Linux 中，某文件执行 `chmod 640 f` 后，再执行 `chmod g+x,o+r f`。若仅考虑传统的用户/组/其他三组权限，则文件最终的八进制权限为（ ）

- A. 650
- B. 654
- C. 644
- D. 754

2. 一棵哈夫曼树采用二叉链表存储。若整棵树中共有 254 个非空孩子指针，则该哈夫曼树的叶子结点数为（ ）

- A. 127
- B. 128
- C. 129
- D. 255

3. 对一棵包含 n 个结点的完全二叉树，对每个叶子结点都从该结点开始不断访问父结点，直到根结点为止。若访问一个结点记为一次基本操作，则全部操作次数的渐近复杂度为（ ）

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(\log n)$

4. 小 h 从 $(0,0)$ 出发，每一步只能向右或向上走，到达 $(8,6)$ ，整个过程中始终满足 $x \geq y$ ，且不能经过点 $(4,2)$ 。满足条件的路径共有（ ）条

- A. 441
- B. 442
- C. 443
- D. 444

5. 用两个栈 A、B 模拟一个队列：入队时总是压入 A；出队时若 B 为空，则把 A 中全部元素依次弹出并压入 B，再从 B 弹出一个元素；若 B 非空则直接从 B 弹出。初始两栈均为空，依次执行

push(1), push(2), pop(), push(3), push(4),
pop(), push(5), pop(), pop()。

整个过程中, 元素从 A 转移到 B 的次数为 ()

- A. 4
- B. 5
- C. 6
- D. 7

6. 将整数 $1, 2, \dots, 12$ 按顺序排成一个圆环 (因此 1 与 12 也相邻)。从中选出 4 个不同的数, 要求任意两个被选数在圆环上都不相邻。不同的选法共有 ()

- A. 84
- B. 96
- C. 105
- D. 126

7. 执行下列 C++ 代码后, 输出为 ()

```
01 map<int, int> mp;  
02 mp[2] = 1;  
03 mp[1] = mp[2] + 2;  
04 int x = mp[3];  
05 mp[2] += mp[1];  
06 cout << mp.size() << ' ' << mp[1] + mp[2] + mp[3];
```

- A. 2 4
- B. 2 7
- C. 3 4
- D. 3 7

8. 对一个有向无环图进行 Kahn 拓扑排序。下列条件与“该图的拓扑序唯一”等价的是 ()

- A. 每个结点的出度都不超过 1
- B. 算法的每一步中, 当前入度为 0 且尚未删除的结点恰好有 1 个
- C. 图中恰好有 $n-1$ 条边
- D. 从任意结点开始 DFS 得到的完成时间序列都相同

9. 对模式串 ababaabab 计算 KMP 的前缀函数 (即 $pi[i]$ 为前缀 $s[0..i]$ 的最长真前后缀长度), 则最后一个位置的前缀函数值为 ()

- A. 2
- B. 3
- C. 4
- D. 5

10. 对一个 n 个顶点、 m 条非负权有向边的图, 使用邻接矩阵实现、不使用堆优化的 Dijkstra

算法求单源最短路。无论 m 的大小如何，其最坏时间复杂度为（ ）

- A. $\Theta(n)$
- B. $\Theta(m \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(nm)$

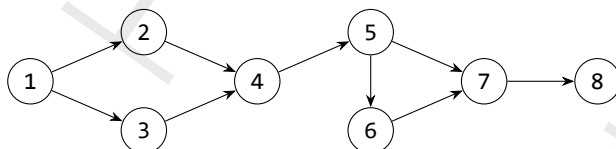
11. 已知 $p=20260913$ 为质数。在模 p 意义下，记除法为乘以逆元，则 $\sum_{i=1}^{p-2} \frac{1}{i(i+1)} \pmod{p}$ 的值为（ ）

- A. 0
- B. 1
- C. 2
- D. $p-2$

12. 在线段 $[0, 2]$ 上独立、均匀随机取 3 个点。包含这 3 个点的最短线段长度（即最大坐标减最小坐标）的期望为（ ）

- A. $2/3$
- B. 1
- C. $4/3$
- D. $3/2$

13. 下图为一个有向图。恰好删除两条不同的边后，仍要求存在从 1 到 8 的有向路径，则不同的删边方案共有（ ）种



- A. 12
- B. 15
- C. 18
- D. 21

14. 有一堆 2026 颗石子，两人轮流操作，每次必须从中取走恰好 1、3 或 4 颗石子中的一种；无法操作的人判负。双方都采用最优策略，则先手第一步有多少种取法可以使后手面对一个必败态（ ）

- A. 0
- B. 1
- C. 2
- D. 3

15. 普通 Nim 游戏中有四堆石子，数量依次为 3, 4, 5, 8。两人轮流选择一堆并取走至少 1 颗石子，可以一次取完；无法操作的人判负。双方采用最优策略，则下列哪一步可以使先手立即把局面变为必败态（ ）

- A. 从第 1 堆取走 2 颗
- B. 从第 2 堆取走 3 颗
- C. 从第 3 堆取走 3 颗
- D. 从第 4 堆取走 6 颗

二、阅读程序（判断题正确填 √，错误填 ×；判断题每题 1.5 分，普通单选题每题 3 分，第 33 题 4 分，共计 40 分）

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, a[N], tmp[N];
05long long ans;
06void merge_sort(int l, int r) {
07    if (l >= r)
08        return;
09    int mid = (l + r) >> 1;
10    merge_sort(l, mid);
11    merge_sort(mid + 1, r);
12    int i = l, j = mid + 1, k = l;
13    while (i <= mid && j <= r) {
14        if (a[i] <= a[j])
15            tmp[k++] = a[i++];
16        else {
17            tmp[k++] = a[j++];
18            ans += mid - i + 1;
19        }
20    }
21    while (i <= mid)
22        tmp[k++] = a[i++];
23    while (j <= r)
24        tmp[k++] = a[j++];
25    for (int p = l; p <= r; ++p)
26        a[p] = tmp[p];
27}
28int main() {
29    ios::sync_with_stdio(0);
30    cin.tie(0);
31    cin >> n;
32    for (int i = 1; i <= n; ++i)
33        cin >> a[i];
```

```

34     merge_sort(1, n);
35     cout << ans;
36 }

```

• 单选题

16. 若初始数组为 [4,1,3,2,5], 只调用 merge_sort(1,4), 调用返回后 a[1..5] 为 ()

- A. [1,2,3,4,5]
- B. [1,2,4,3,5]
- C. [1,3,2,4,5]
- D. [4,1,3,2,5]

• 判断题

17. 若调用前 ans=0, 调用 merge_sort(1,n) 后, ans 等于原数组的逆序对数量。()

- A. 正确
- B. 错误

18. 对于下标满足 $i < j$ 且 $a[i] = a[j]$ 的两个元素, 这一对元素会对 ans 贡献 1。()

- A. 正确
- B. 错误

19. 若把 ans 的类型改成 32 位有符号 int, 则在题目给定的数组上界 $n=2e5$ 下可能发生溢出。()

- A. 正确
- B. 错误

• 单选题

20. 当输入数组为 [3,1,2,2,1,3] 时, 程序最终输出为 ()

- A. 5
- B. 6
- C. 7
- D. 8

21. 当 $n=8$, 且每次合并时两个有序区间的元素比较次数都达到可能的最大值, 则所有合并阶段执行 $a[i] \leq a[j]$ 的总次数为 ()

- A. 12
- B. 14
- C. 17
- D. 24

```

01#include <bits/stdc++.h>
02using namespace std;
03int main() {
04     ios::sync_with_stdio(0);

```

```

05     cin.tie(0);
06     int n, k;
07     string s;
08     cin >> n >> k >> s;
09     vector<char> st;
10     st.reserve(n);
11     for (char ch : s) {
12         while (k > 0 && !st.empty() && st.back() >
ch) {
13             st.pop_back();
14             --k;
15         }
16         st.push_back(ch);
17     }
18     while (k > 0) {
19         st.pop_back();
20         --k;
21     }
22     for (char ch : st)
23         cout << ch;
24 }

```

• 判断题

22. 当初始 $k=0$ 时，程序扫描字符串的过程中不会执行任何一次 `pop_back()`。()

- A. 正确
- B. 错误

• 单选题

23. 输入为 9 4 cbacdcba。扫描完前 6 个字符后，st 从底到顶的内容以及此时 k 的值分别为 ()

- A. acd, 2
- B. acc, 1
- C. acb, 0
- D. abc, 1

• 判断题

24. 扫描完整个字符串后，如果仍有 $k>0$ ，继续从当前序列末尾删除字符可以得到字典序最小的结果。()

- A. 正确

B. 错误

• 单选题

25. 当输入为 8 3 dcabccba 时, 扫描字符串的过程中 st 的最大长度为 ()

A. 3

B. 4

C. 5

D. 6

26. 当输入为 8 3 dcabccba 时, 程序的最终输出为 ()

A. abcba

B. abccb

C. acbba

D. bccba

• 判断题

27. 虽然程序中存在嵌套的 while, 但每个字符最多入栈一次、出栈一次, 因此程序的总时间复杂度为 $O(n)$ 。()

A. 正确

B. 错误

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, head[N], to[N * 2], nxt[N * 2], ec;
05int dista[N], que[N];
06void add(int u, int v) {
07    to[++ec] = v;
08    nxt[ec] = head[u];
09    head[u] = ec;
10}
11int bfs(int start) {
12    fill(dista + 1, dista + n + 1, -1);
13    int l = 0, r = 0, far = start;
14    que[r++] = start;
15    dista[start] = 0;
16    while (l < r) {
17        int u = que[l++];
18        if (dista[u] > dista[far])
19            far = u;
```

```

20     for (int e = head[u]; e; e = nxt[e]) {
21         int v = to[e];
22         if (dista[v] == -1) {
23             dista[v] = dista[u] + 1;
24             que[r++] = v;
25         }
26     }
27 }
28 return far;
29 }
30 int main() {
31     ios::sync_with_stdio(0);
32     cin.tie(0);
33     cin >> n;
34     for (int i = 1, u, v; i < n; ++i) {
35         cin >> u >> v;
36         add(u, v);
37         add(v, u);
38     }
39     int s = bfs(1);
40     int t = bfs(s);
41     cout << dista[t];
42 }

```

• 单选题

28. 若输入边依次为 1-2,1-3,2-4,2-5,3-6,3-7, 则调用 bfs(1) 返回的结点为 ()

- A. 4
- B. 5
- C. 6
- D. 7

• 判断题

29. 若 BFS 中有多个结点与起点距离同为最大值, 则函数会保留这些结点中最先从队列里取出的那个作为 far。()

- A. 正确
- B. 错误

30. 对同一棵树, 仅改变输入边的先后顺序, 第一次调用 bfs(1) 的返回值可能发生改变。()

- A. 正确

B. 错误

31. 对任意一棵树，从任意结点进行一次 BFS 得到某个最远点 s ，再从 s BFS 得到最远点 t ，则 $\text{dista}[t]$ 等于该树的直径长度。()

A. 正确

B. 错误

32. 若把输入从“树”放宽为任意连通无权图，则仍可保证上述“两次 BFS”方法得到图的直径。()

A. 正确

B. 错误

• 单选题

33. (本题 4 分) 若输入的树有边 1-2, 1-3, 2-4, 2-5, 3-6, 3-7，程序最终输出为 ()

A. 2

B. 3

C. 4

D. 5

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (有序序列的第 k 小和) 有两个长度均为 n 的单调不降序列 a 和 b 。从两个序列中各取一个数相加，共得到 n^2 个和，求这些和中第 k 小的值。输入满足 $1 \leq n \leq 10^5$, $1 \leq k \leq n^2$ ，元素为非负整数且答案不超过 10^9 。程序用双指针在线性时间内计算“不超过某个值的数对数量”。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 1e5 + 5;
04int n, a[N], b[N];
05long long k;
06long long count_le(long long x) {
07    long long cnt = 0;
08    int j = ①;
09    for (int i = 1; i <= n; ++i) {
10        while (②)
11            --j;
12        cnt += ③;
13    }
14    return cnt;
15}
16long long solve() {
17    long long l = ④;
18    long long r = 1LL * a[n] + b[n];
19    while (l + 1 < r) {
20        long long mid = (l + r) >> 1;
21        if (count_le(mid) >= k)
22            r = mid;
23        else
24            l = mid;
25    }
26    return ⑤;
27}
28int main() {
29    ios::sync_with_stdio(0);
30    cin.tie(0);
31    cin >> n >> k;
```

```

32     for (int i = 1; i <= n; ++i)
33         cin >> a[i];
34     for (int i = 1; i <= n; ++i)
35         cin >> b[i];
36     cout << solve();
37 }

```

34. ①处应填 ()
 A. n B. n-1
 C. 1 D. 0
35. ②处应填 ()
 A. $j > 0 \ \&\& \ 1LL * a[i] + b[j] > x$
 B. $j > 0 \ \&\& \ 1LL * a[i] + b[j] \geq x$
 C. $j < n \ \&\& \ 1LL * a[i] + b[j] > x$
 D. $j > 0 \ \&\& \ 1LL * a[i] + b[j] \leq x$
36. ③处应填 ()
 A. j B. n-j
 C. j+1 D. n
37. ④处应填 ()
 A. $1LL * a[1] + b[1]$
 B. 0
 C. $1LL * a[1] + b[1] - 1$
 D. $1LL * a[n] + b[n]$
38. ⑤处应填 ()
 A. 1 B. r
 C. l+1 D. count_le(r)

(2) (减法游戏与 SG 函数) 有 n 堆石子和 m 种允许的取石数量 $d[1..m]$ 。每次操作选择一堆, 并从中恰好取走某个 $d[j]$ 颗石子 (该堆石子数必须不少于 $d[j]$); 无法操作的人判负。各堆互不影响, 双方均采用最优策略。输入满足 $1 \leq n \leq 10^5$, $1 \leq m \leq 20$, $0 \leq a[i] \leq 10^5$ 。下面程序计算每个石子数的 SG 函数, 并判断先手是否必胜。

```

01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 1e5 + 5;
04 int n, m, d[25], a[N], sg[N];
05 int vis[25], tim;
06 int main() {

```

```

07 ios::sync_with_stdio(0);
08 cin.tie(0);
09 cin >> n >> m;
10 for (int i = 1; i <= m; ++i)
11     cin >> d[i];
12 int mx = 0;
13 for (int i = 1; i <= n; ++i) {
14     cin >> a[i];
15     mx = max(mx, a[i]);
16 }
17 for (int x = 1; x <= mx; ++x) {
18     ++tim;
19     for (int j = 1; j <= m; ++j) {
20         if (①)
21             vis[②] = tim;
22     }
23     int g = 0;
24     while (③)
25         ++g;
26     sg[x] = ④;
27 }
28 int xr = 0;
29 for (int i = 1; i <= n; ++i)
30     xr ^= ⑤;
31 cout << (xr ? "Win" : "Lose");
32 }

```

39. ①处应填 ()

- A. $x \geq d[j]$
- B. $x > d[j]$
- C. $x \leq d[j]$
- D. $x - d[j] < 0$

40. ②处应填 ()

- A. $sg[x-d[j]]$
- B. $sg[x+d[j]]$
- C. $x-d[j]$
- D. $d[j]$

41. ③处应填 ()
- A. `vis[g] == tim`
 - B. `vis[g] != tim`
 - C. `g < m`
 - D. `sg[g] == tim`
42. ④处应填 ()
- A. `g` B. `g+1`
 - C. `tim` D. `vis[g]`
43. ⑤处应填 ()
- A. `sg[a[i]]`
 - B. `a[i]`
 - C. `d[i]`
 - D. `sg[i]`